Introduction to Wireshark

Contents

About 2
Installation 2
Tutorial 3
- Capture Interface 3
- Stop / Restart Capture 6
- Filters 6
  - Capture Filters 7
  - Display Filters 9
  - Capture / Display Filter Language 10
  - “Display Filter Expression” Function 11
  - “Apply/Prepare as Filter” Function 12
- Capture File 14

Extra Resources 15
- Marking Packets 15
- Packet Comments 15
- Keyboard Shortcuts 16
- Useful Links 16

About

Quoting the official FAQ: "Wireshark is a network protocol analyzer. It lets you capture and interactively browse the traffic running on a computer network."

In this course, we use it to learn the workings of network protocols. In real life, it is used for network troubleshooting, network security analysis, and many other purposes. It is a powerful tool, while free and open source, thus hugely popular!

Installation

Wireshark captures traffic by receiving a copy of messages from and to your computer in the form of *packets*. It uses the *libpcap* packet capture library which would be installed together with Wireshark if it is not already found on your computer.

On Windows or macOS: Download and run the Wireshark installer from https://www.wireshark.org/download.html.

For other UN*X operating systems: Download the source code from the same link and set up, build and install Wireshark based on the instructions here: https://www.wireshark.org/docs/wsdg_html_chunked/ChSetupUNIX.html.

You may keep to the default selected options for the installation. In particular:

- Npcap is required.
- USBPcap is not required.

Refer to https://www.wireshark.org/docs/wsug_html/#ChapterBuildInstall if you run into any problems with the installation process.

Tutorial

Go through this tutorial to familiarize yourself with the core features of Wireshark.

The actions for you to follow are placed in a green box, like this one.

This tutorial includes high-level explanation of related concepts to give context to the Wireshark functions. We will learn the concepts in more depth in the course, so do not worry too much about them here.

Some tips as you go through the tutorial:

- Keyboard shortcuts are very useful, but these differ across operating systems, hence for simplicity the instructions are based on navigating the Wireshark menu to accomplish things. It is highly recommended that you note the keyboard shortcuts indicated on the menu while following the instructions. We list some of them at the end of this document as well.
- Each topic in the tutorial is accompanied with the link to the documentation, mainly as an additional resource if you want to learn more or run into unexpected behaviours. We try to include the more salient information in the tutorial itself, and highlight the few documentation pages that we think you should take time to read.

Capture Interface

[Documentation]

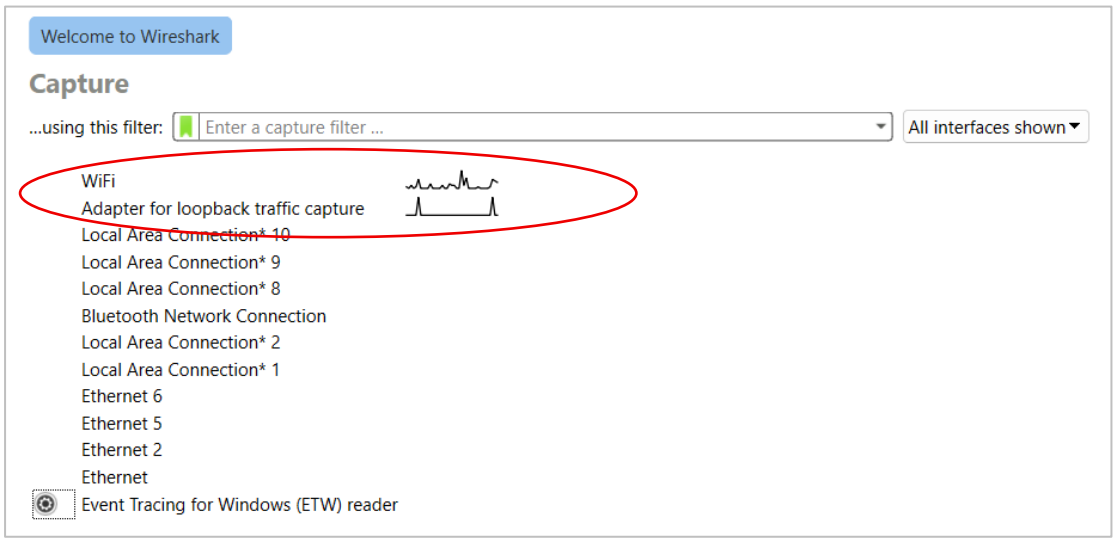
Start Wireshark. You will be greeted by the interface selection screen (see *Figure 1*).

Connecting a computer to a network is accomplished by hardware and software functions that together can be referred to as a **network interface**. Modern computers are often equipped with multiple network interfaces to connect to various network types (e.g., Ethernet, Bluetooth).

To capture packets for analysis, Wireshark needs to know which network interface to capture from. It lists all available interfaces that you can select, along with detected network activity[1] on each, shown as a sparkline next to the interface name (circled in red in *Figure 1*).

[1] In case you are on UN*X and none of the interfaces have network activity, you may need to set up capture privileges. See: https://wiki.wireshark.org/CaptureSetup/CapturePrivileges#limiting-capture-permission-to-only-one-group.

Figure 1: Capture Interface Selection



Typically, you will see network activity from:

- The network interface connecting your computer to the Internet, e.g., by Wi-Fi, Ethernet (cable), or using a VPN service. *Table 1* lists the common interface names for the different connection types in different OSes.
- **Adapter for loopback traffic capture**: a virtual interface to capture traffic from the computer to itself. This is useful, for example, when you are testing a web application locally, with both web server and application running on the same computer. In this setup, the exchanged messages are within the same computer, not passing through a "real" network interface. This virtual interface allows Wireshark to capture them.

Table 1: Common Interface Names in Various Operating Systems

Operating System	Connection Type		
	Wi-Fi	**Ethernet**	**VPN**
Windows	`Wi-Fi`	`Ethernet`	`e.g. McAfee VPN`
macOS	`en[X]`		`utun[X], ppp[X]`
Other UN*X	`wlp[X]`	`enp[X]`	`ppp[X], tun[X], wg[X]`

For this tutorial, let us capture messages from your internet connection.

Select the interface corresponding to how you are connected to the Internet.

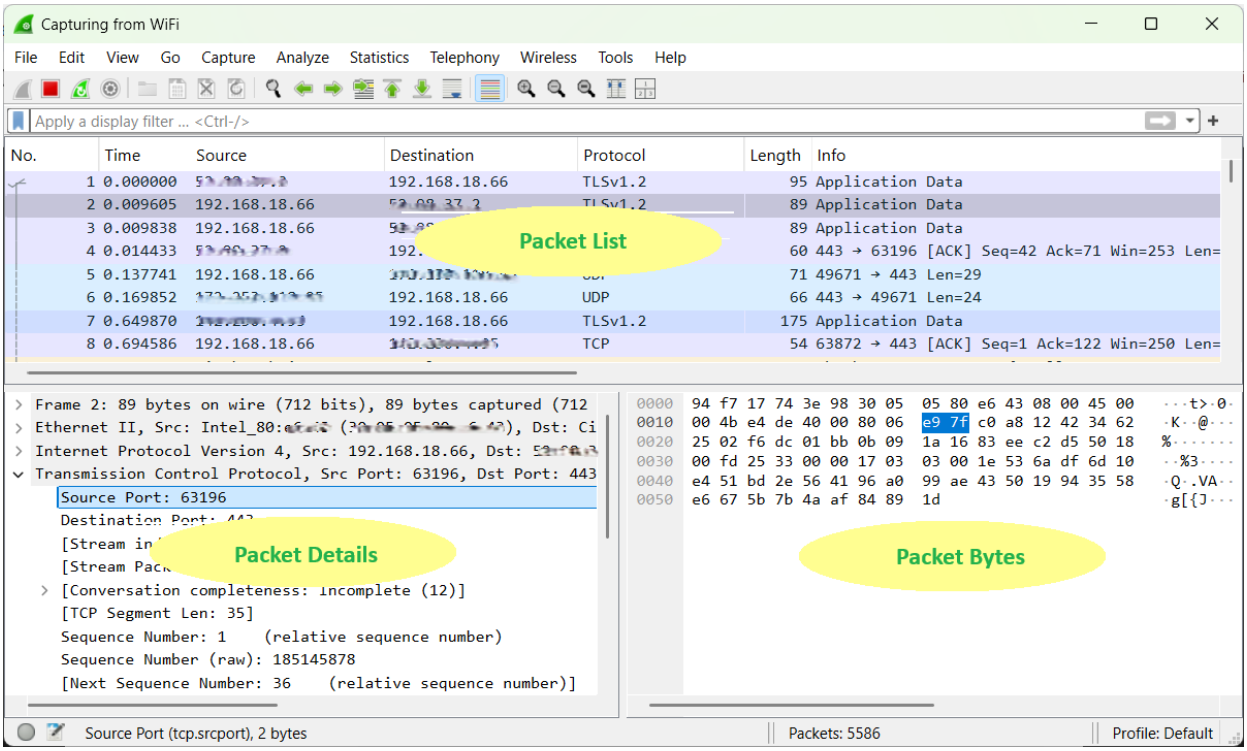
Double-click on the interface name to immediately start capturing. The captured packets will be listed as soon as they are captured.

In the default Wireshark layout, you will see 3 panes (see *Figure 2*):

- **Packet List**: Shows all packets in the current capture, with colour coding. You can select a packet by clicking the corresponding line. For a quick explanation on the columns, see The "Packet List" Pane documentation.
- **Packet Details:** Shows the content of the selected packet in a tree format, representing the *protocol* stack. Each protocol heading can be expanded/collapsed to show/hide its *fields*. To briefly explain, the packet essentially consists of protocol information in the form `<field_name>: <field_value>`. We will learn about protocols in detail as we go through the course.
- **Packet Bytes**: Shows the data of the selected packet in a hexdump style.

Wireshark supports another type of pane, **Packet Diagram**, which shows a visual layout of the packet data. You can change the layout or combination of panes from the menu: **Edit → Preferences → Appearance → Layout**.

Figure 2: Wireshark Layout



Stop / Restart Capture

[Documentation]

You should have a running capture on Wireshark from the previous step in this tutorial.

Stop capturing by clicking on the red square ( ) at the top left corner.

After stopping a capture, you can restart the capture using the same settings. For easier analysis, we usually want a capture to contain as few irrelevant packets as possible. The stop and restart functions make it convenient to isolate the session of our interest, for example, a web application that you are debugging.

For this tutorial, let's capture an isolated network traffic of a visit to the Wireshark website.

Close all web browsers on your computer. If possible, also close any other applications that connect to the Internet. (We will not worry about background processes here.)

Restart the capture on Wireshark by clicking on the blue shark fin icon ( ) at the top left corner. When asked if you want to save the capture, choose "Continue without Saving".

Open a browser and visit the Wireshark website: https://www.wireshark.org/.

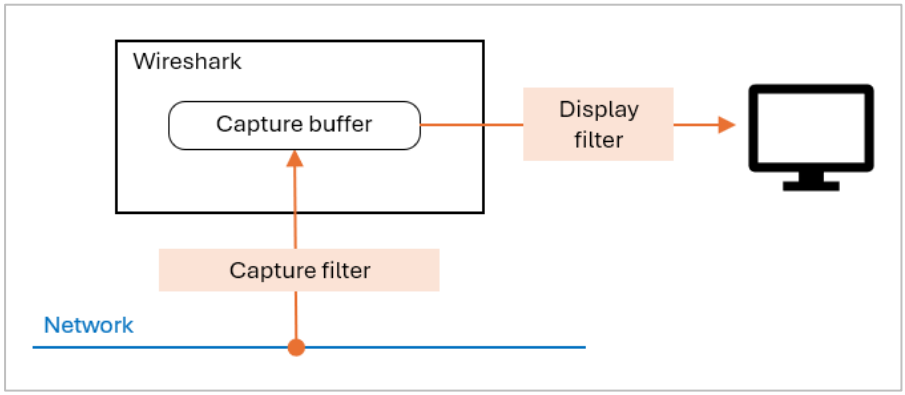
After the webpage has finished loading, stop the capture on Wireshark.

Filters

As you can see, a whole lot of packet traffic might be passing through a computer while it is connected to the network! To make sense of the network traffic, we rely heavily on **filtering** the packets relevant to our purpose.

Filtering is in fact a core function in Wireshark, and it does this in two places: **capture** and **display**. To understand the difference, think of Wireshark as a tool that taps into a pipe that is the network connection, captures (i.e., stores a copy of) the flowing packets into a buffer, and displays the buffer content to us. *Figure 3* gives a simple illustration.

Figure 3: Illustration of Wireshark Filtering



We can set **capture filters** to limit the type of packets to capture. Wireshark will store only those packets into the buffer and ignore other packets passing through the network. After that, we can use **display filters** to display only a subset of the captured packets.

For example, to monitor the connection to a specific server, we may use a capture filter by IP address to capture only the packets coming from or going to that server's address. After that, we may use various display filters to examine one specific context at a time, e.g., by protocol such as DNS or HTTP, by port number, and so on.

The two filter types have different specification languages. Before we delve into them, let's first try our hands on some filtering!

Capture Filters

[Documentation]

Capture filters must be set before we tell Wireshark to start capturing (it only makes sense!). For this example, let's capture only the packets sent from your computer.

First, look up the IP address of your computer, which will identify your computer in the packets. This is a series of four numbers separated by dots, e.g., `192.168.1.1`.

On Windows:

Open the Command Prompt and enter the command: `ipconfig`

Look for the IPv4 address under your network interface.

On macOS or other UN*X systems:

Open the terminal and enter the command: `ifconfig`

Look for your network interface. Your IP address is right after the `inet` tag.

With this on hand, let's proceed with setting the capture filter.

Make sure there is no ongoing capture on Wireshark. On the Wireshark main menu at the top bar, select **Capture → Options**. The Capture Options dialog will appear (see *Figure 4*).

Single-click to select the interface corresponding to your Internet connection.

Optional: Click the > symbol on the left of the interface name to expand it. You should see a list of addresses associated with the interface. Compare the IP address value with what you obtain from the Command Prompt / terminal.

In the input field "Capture Filter for selected interfaces", enter the following string, replacing `<ip_addr>` with your actual IP address:

```
src host <ip_addr>
```

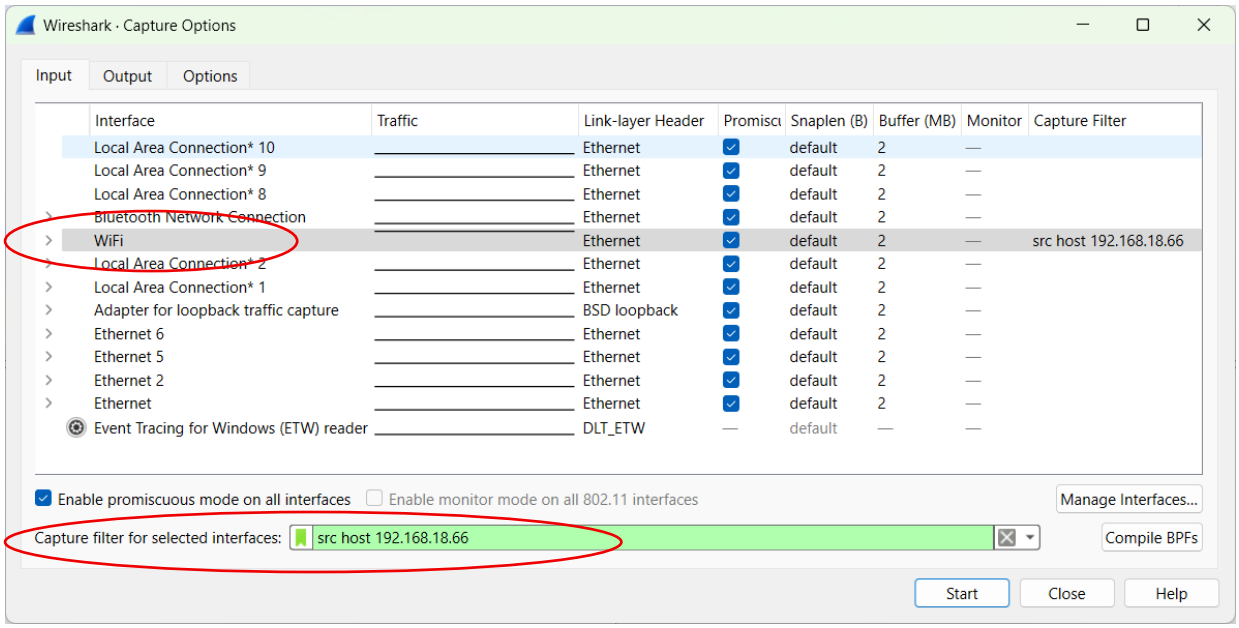
Observe that the input field turns green when you have entered a complete, valid expression for the filter. Click the Start button to start capture.

You can verify the capture filter which is currently active by looking at the status indicated on the Wireshark app title bar (top of the app window).

Refresh the Wireshark website on your browser to make sure to capture its traffic.

The above filter expression basically specifies that we only want packets with your IP address as the originating computer ("source host").

Figure 4: Capture Options Dialog



Additional notes:

- The Capture Filter input field is also present in the interface selection screen when you first start Wireshark (refer to *Figure 1*).
- Restarting a capture (refer to the *Stop / Restart Capture* section) will immediately start a new capture with the same settings (interface selection, capture filter expression if any) as your previous capture. If you need to update the capture filter between captures, be sure to use the **Capture → Options** menu (or the corresponding shortcut) instead.

Display Filters

[Documentation]

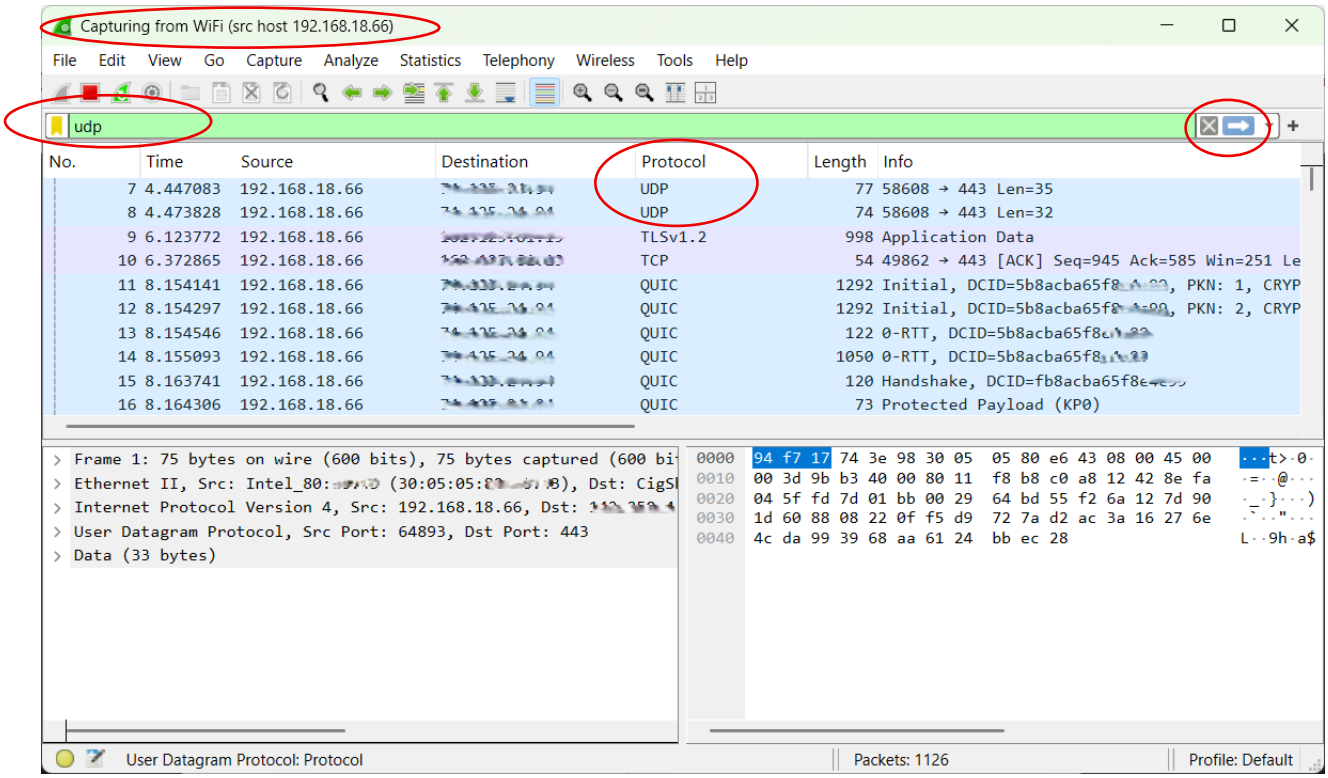
The Display Filter input field is prominently positioned above the Packet List (see *Figure 4*).

Wireshark should have a packet capture running from the previous step of this tutorial. Look at the Protocol column and note the protocol value of any packet (e.g., "UDP").

Enter the protocol value in its lowercase form (e.g. udp) in the Display Filter input field.

Press Enter (or click the arrow button beside the field) to apply the filter. The Packet List pane will be updated to show only packets corresponding to this protocol.

Figure 5: Entering a Display Filter



Observe values in the Protocol column with the display filter applied. Do you see values other than the specified protocol? If so, can you find out why from the Packet Details? (We will discuss more about this in the course!)

Unlike a capture filter which is specified once before capturing, we can freely add, change, or clear display filters during a capture (i.e., while "live" capturing is going on) as well as after a capture (i.e., applied on the captured packets after stopping the capture).

Stop the capture.

Change the display filter to a different protocol (choose another value in the Protocol column).

Clear the filter by clicking the cross button ( ) beside the Display Filter input field.

Capture / Display Filter Language

Capture filters and display filters are specified in different languages, which are only roughly and incompletely described here. You are not expected to learn the specifications in depth, but you should try to be familiar with frequently used filters and know how to find the filter expressions you need.

The following operators work on both capture and display filters:

- Comparison: `<`, `<=`, `>`, `>=`, `!=`, `==`
- Boolean: `&&` (and), `||` (or), `!` (not)
- Parentheses: `( )`

Other operators may be supported within the specific filter languages.

Capture filters: Based on libpcap filter language, these are built from expressions, often in the form `<terms> <value>`, possibly combined with `and` / `or` / `not` / parentheses. It is best to look at examples to get an idea of how they work.

- *Table 2* lists some commonly used capture filters.
- The Wireshark menu **Capture → Capture Filters...** is where you can save a list of your frequently used capture filters. It comes pre-populated with examples that you can use or copy-and-modify. Do try them out!
- The Capture Filters documentation provides more examples, with explanation.

Display filters: Specified in Wireshark's own filter language, this could be:

- a protocol name, or
- an expression in the form `<packet_field> <comparison> <value>`, or
- expressions combined with `and` / `or` / `not` / parentheses.

Do take some time to read the Building Display Filter Expressions documentation to get an overall understanding. Also see examples in *Table 2* and the Display Filters documentation.

Table 2: Common Capture / Display Filter Expressions

Filter by	Details	Capture Filter	Display Filter
IP address	Any direction	`host <ip_addr>`	`ip.addr == <ip_addr>`
	From	`src host <ip_addr>`	`ip.src == <ip_addr>`
	To	`dst host <ip_addr>`	`ip.dst == <ip_addr>`
Port number	Any direction	`port <port_num>`	`tcp.port == <port_num> ||` `udp.port == <port_num>`
	From	`src port <port_num>`	`tcp.srcport == <port_num> ||` `udp.srcport == <port_num>`
	To	`dst port <port_num>`	`tcp.dstport == <port_num> ||` `udp.dstport == <port_num>`

Filter by	Details	Capture Filter	Display Filter
Frame properties	Length limit	`len > <byte_count>`	`frame.len > <byte_count>`
	Time limit	*(Use Capture → Options, Options tab: "Stop capture automatically after...")*	`frame.time_relative <= <duration_in_seconds>`
String / Bytes search	Entire packet		`frame contains <string_or_bytes>`
	Protocol payload		`<protocol> contains <string_or_bytes>`
	Protocol field		`<protocol>.<field> contains <string_or_bytes>`

For display filters, Wireshark provides functionalities to compose them without typing the expressions from scratch. Using these is also a great way to learn the expressions! We will look at these in the next sections.

"Display Filter Expression" Function

This Wireshark function lets you search and select from all valid fields and operators to compose a display filter.

Suppose you have heard that when you visit a website over HTTPS, some sort of "handshake" and the TLS protocol are involved. Without knowing the details, you just want to look up the packets involved in this handshake to examine them.

From the previous step in this tutorial, Wireshark should have a filtered capture of packets originating from your computer while browsing the Wireshark website.

Ensure that the Display Filter input field is empty. (The function will append to this field, hence we want to start out empty.)

From the Wireshark main menu at the top bar, select **Analyze → Display Filter Expressions...**. The Display Filter Expression dialog will appear (see *Figure 6*).

Search for "handshake" in the search box. The Field Name pane will be updated with a narrowed down list of protocols with field names that match it.

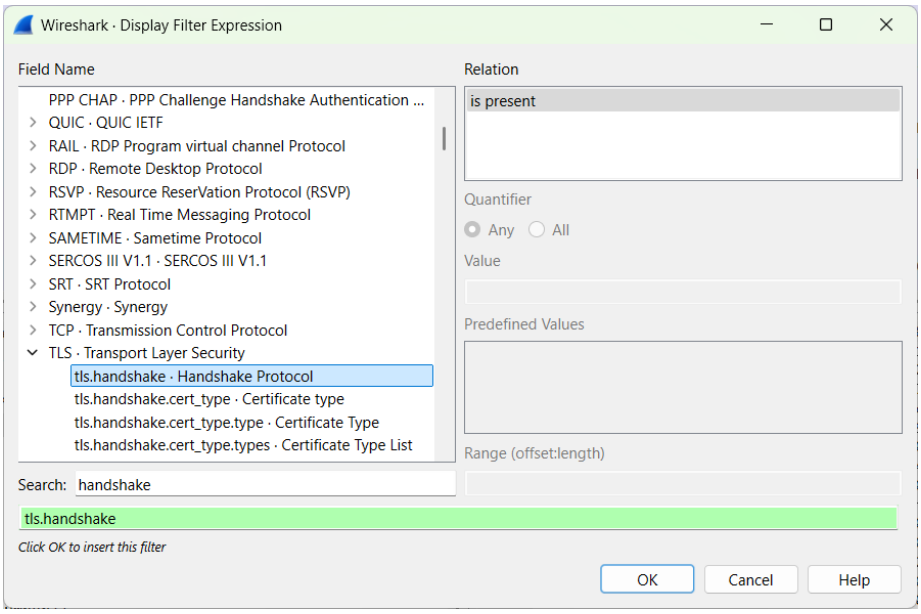
Scroll down until you find "TLS", then expand the heading.

Click on `tls.handshake` in the expanded list. The text will appear in the field at the bottom.

Click the OK button. The expression will be inserted into the Display Filter input field (but not yet applied).

Apply the filter by clicking , or by clicking on the Display Filter input field and pressing Enter.

Figure 6: Display Filter Expression Functionality



"Apply/Prepare as Filter" Function

This Wireshark function is useful if you have identified a packet with the feature (i.e., field values) that matches how you want to filter. You can go to that specific field in that packet and tell Wireshark to use it as a display filter. There are two flavours to this:

- **Apply as Filter** will enter the corresponding expression in the Display Filter field and immediately apply it.
- **Prepare as Filter** will only enter the corresponding expression in the Display Filter field, giving you a chance to modify it before applying it manually.

A small detail: Unlike the Display Filter Expression function discussed earlier, these functions will replace the existing expression (if any) in the Display Filter field, instead of appending to it.

Suppose you have found a packet that your computer sent to the Wireshark website server, and you know which *process* in your computer was responsible for it by looking at the source *port number* in that packet (we will learn more about these concepts in the course). You want to look up all other packets sent from that process.

First, let's see how we can search packets for specific details in Wireshark. Here, we will do a string search for "wireshark.org", the Wireshark website address.

From the previous step in this tutorial, Wireshark should have a filtered capture of packets originating from your computer while browsing the Wireshark website.

From the Wireshark main menu, select **Edit → Find Packet...**. This will call up a Search input field between the Display Filter input field and the Packet List pane (see *Figure 7*).

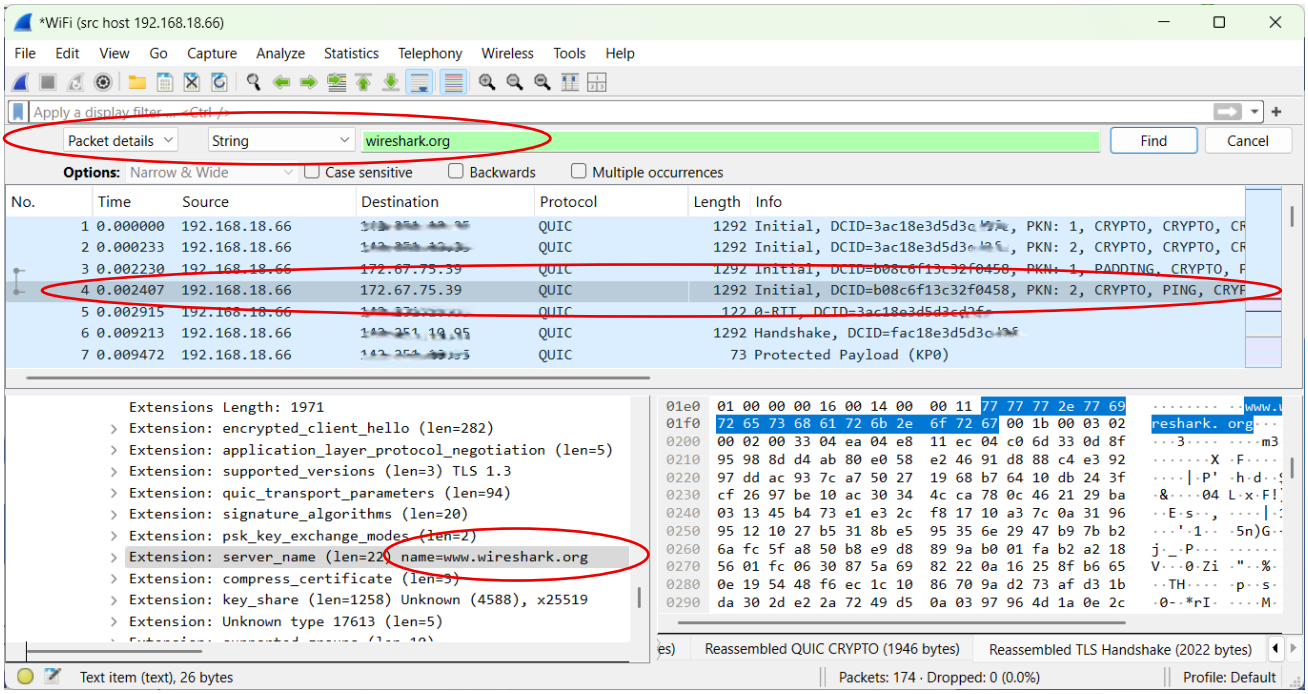
Make sure the search setting is set to "Packet details" and "String".

Type “wireshark.org” in the Search input field and press Enter. The first matching packet, if any, will be selected and highlighted in grey in the Packet List. The Packet Details pane will also be updated with this selected packet.

You can press Enter or click the Find button to jump to the next matching packet. Keep an eye on the search result status, shown at the bottom left of the Wireshark app window.

If you encounter an empty search result for the above steps, you may select an arbitrary packet (e.g., look for any UDP packet) to continue with the remaining steps.

Figure 7: String Search in Wireshark



Now that we have identified a packet, let’s proceed with using it to compose a display filter. Here, we want a filter based on the source port number. Assuming this is an UDP packet, the port number will be a field under the “User Datagram Protocol” heading.

The Packet Details pane should be showing the selected packet, highlighting the field that matches the search.

Scroll up the pane to find the heading “User Datagram Protocol”. This will likely be near the top of the pane.

Expand the heading and find the “Source Port” field. The field value is the port number that we want to apply as a filter.

Right-click on this field and select **Apply as Filter → Selected**. You should see the expression entered in the Display Filter input field, along with the updated Packet List.

We can use the field as a **Selected** filter or **Not Selected** filter (prepending `not`).

This function also allows us to build a combination of expressions by joining a new expression to the existing display filter with logical operators `and` / `or` / `and not` / `or not`.

Under the expanded "User Datagram Protocol" heading, find the "Destination Port" field.

Right-click on this field and select **Prepare as Filter → ...and Selected**. You should see the Display Filter input field updated with the combined expressions.

Apply the filter by clicking , or by clicking on the Display Filter input field and pressing Enter.

Capture File

[Documentation]

We can save the captured packets in a capture file. One useful scenario for this is when a user encounters possibly-network-related issues (e.g., terrible lag, unreachable service) with a web application when running it on their computer. The user could capture the network traffic while the application is running, then share the capture file with the developer or a network analyst to analyse and troubleshoot the issue.

Wireshark's native file format, and the default format used for saving, is pcapng.

From the previous step in this tutorial, Wireshark should have a filtered capture of packets originating from your computer while browsing the Wireshark website.

Make sure Wireshark has stopped capture.

From the main menu, select **File → Save**. Provide a name and select a location to save the pcapng file on your computer.

To open the file you have saved, select **File → Open**. Locate the file and click the Open button.

You should also be able to open a pcapng file directly from your file explorer application.

Close Wireshark.

Locate the file on your computer and double-click to open it. This should trigger Wireshark to launch and open the file.

Naturally, Wireshark can only save what it has stored into the buffer (refer to *Figure 2*), which means the packets filtered by the capture filter (if any). Thus capture filters are useful to reduce the size of the capture file as well.

We have come to the end of the tutorial. Congratulations on completing it!

The rest of this document contains extra resources you may refer to as you continue to explore Wireshark throughout the course. Feel free to share tips and/or post questions on Wireshark in the Canvas discussion forum too!

Extra Resources

Marking Packets

[Documentation]

You can mark packets in the Packet List to make it easier to find them later. They will be shown with black background.

- Select the packets you want to mark in the Packet List. Multi-select with Shift or Ctrl / Cmd is supported.
- With the packets selected, select **Edit → Mark/Unmark Selected** from the main menu.

You can also mark packets one at a time by simply middle-clicking on the packet, or by right-clicking and selecting Mark/Unmark Selected.

To jump to marked packets, use **Edit → Next Mark** from the main menu.

To view only marked packets, set the display filter to `frame.marked == 1`.

Packet markings are not stored in the capture file. As an alternative, you can save marked packets to a separate capture file using **File → Export Specified Packets...** with the option "Marked packets only".

Packet Comments

[Documentation]

You can add comments to packets, which will be saved in the capture file. To do this, right-click on the packet and select **Packet Comments → Add New Comment...**.

You will see the packet comments in the Packet Details pane.

Keyboard Shortcuts

Use keyboard shortcuts to speed up your Wireshark workflow.

- Toggle packet capture **start/stop**: Ctrl (or Cmd) + E
- **Open** a capture file: Ctrl (or Cmd) + O
- **Save** a capture file: Ctrl (or Cmd) + S
- Go to the **next** packet: Ctrl (or Cmd) + N
- Go **back** one packet: Ctrl (or Cmd) + B
- **Search**: Ctrl (or Cmd) + F
- **Mark/unmark** selected packets: Ctrl (or Cmd) + M
- **Mark all** displayed packets: Ctrl (or Cmd) + Shift + M
- **Unmark all** displayed packets: Ctrl (or Cmd) + Alt + M
- Go to the **next marked** packet: Ctrl (or Cmd) + Shift + N
- Go **back** one **marked** packet: Ctrl (or Cmd) + Shift + B
- Add/edit packet **comment**: Ctrl (or Cmd) + Alt + C

Useful Links

- Official user guide: https://www.wireshark.org/docs/wsug_html_chunked
- Wireshark Tutorial for Beginners, by Anson Alexander: https://youtu.be/qTaOZrDnMzQ